

CMAA5043 Creative Prototyping

Lab 2 Tutorial: Setup React JS & Basics

Welcome to ***Creative Prototyping: Designing the Interactive Interface!*** In this course, we will learn how to quickly design interactive interfaces using **React JS**.

Task 1. Setting up environment: Terminal

The terminal is a tool for developers to interact with the computer's operating system. When developing React applications, the terminal is used to run commands, start the development server, install dependencies, and more.

1.1 Open Terminal

In Windows, you can search for `Terminal` and find it in the results. If you can't search it, you can also search for `PowerShell`, which is a command-line tool in the Windows system. Its functionality is similar to Terminal and can be used to execute various commands.

| In this tutorial, all operations in the Terminal can also be applied in PowerShell.

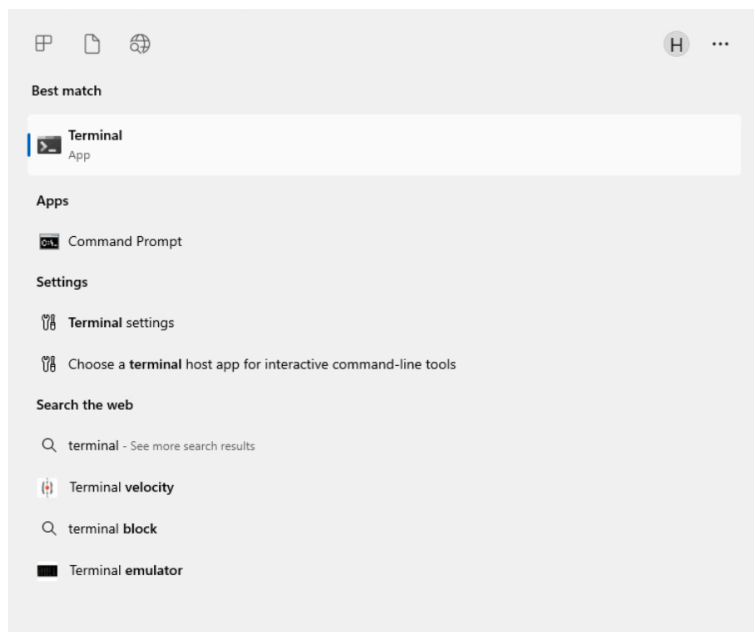


Figure 1: Terminal in search.

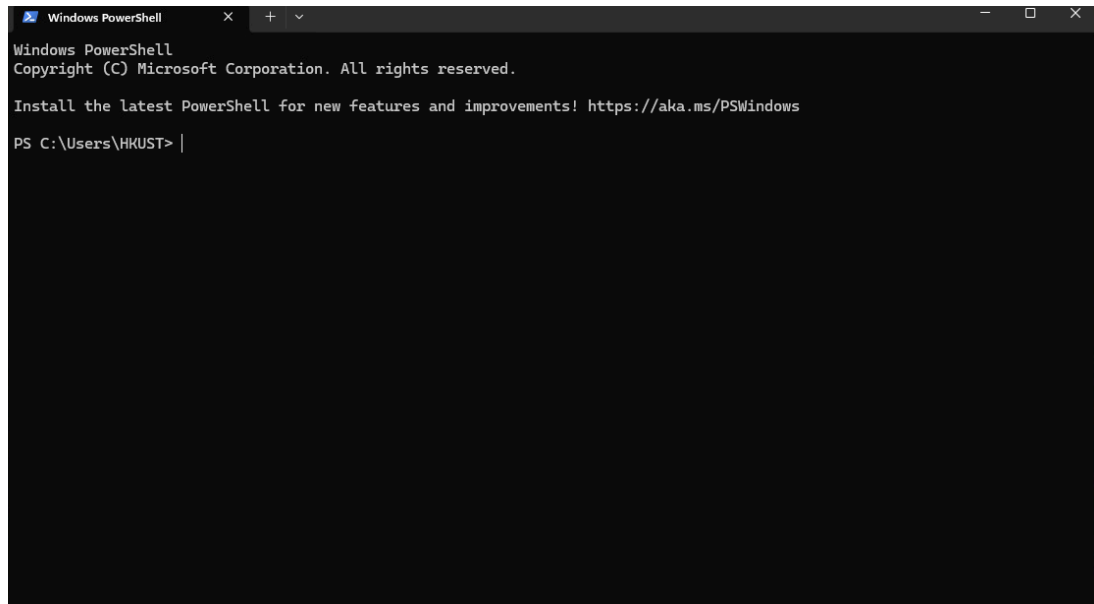


Figure 2: Terminal interface.

1.2 Open File through Terminal

Use the command `cd` followed by the file path to enter the desired folder. For example:

`C:\Users>HKUST> cd C:\[filename]`

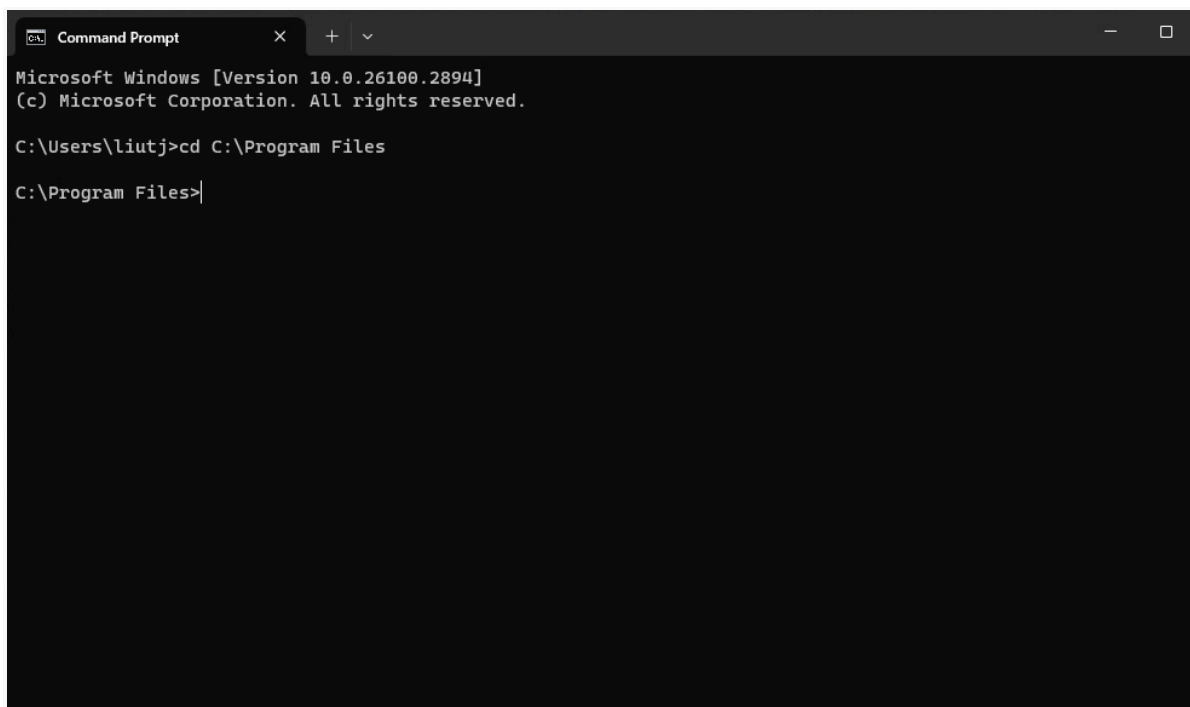


Figure 3: Open Files in Terminal.

You can also right-click in the folder, and you will see the option to `Open in Terminal`. If it's not there, you can click `Show more options`, and you will find the option.

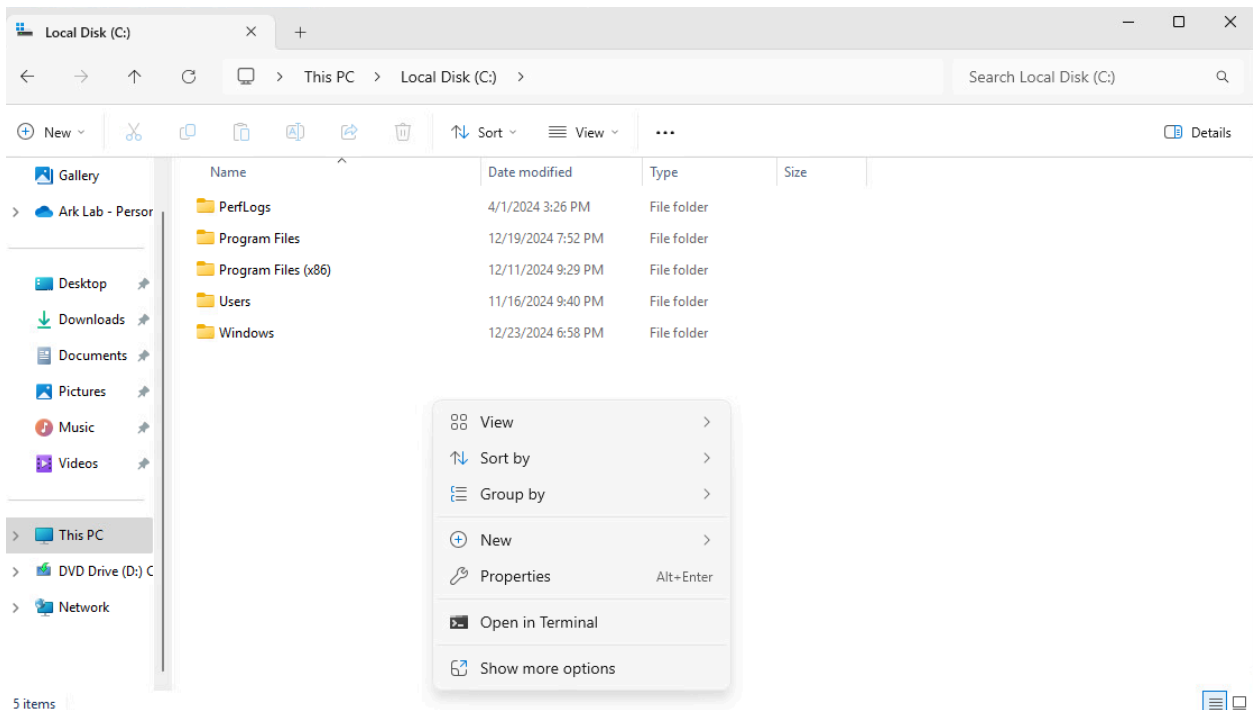


Figure 4: Open terminal in files.

If you want to access a specific project, first navigate to the storage drive (e.g., D: test). This can be done by simply typing `[drive letter:]` and pressing the Enter key.

```
C:\Users>HKUST> D:
```

```
D:> cd D:\test
```

```
D:\test>
```

1.3 Open Terminal as administrator

Sometimes, we may encounter a `Permission Denied` error when executing commands in the Terminal. To resolve this issue, we can right-click on the Terminal and select `Run as administrator`. This will grant the necessary permissions to execute the commands successfully.

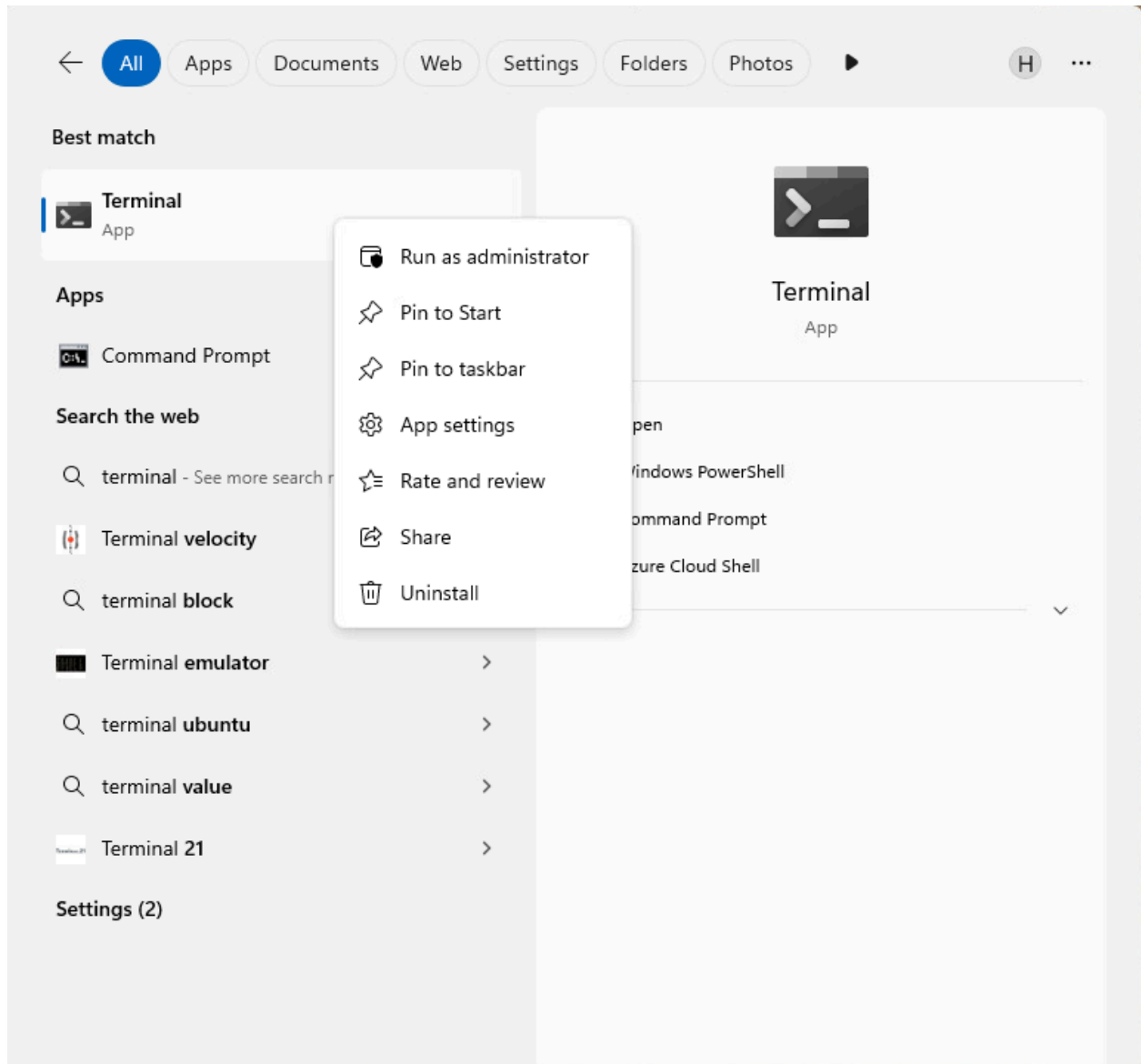


Figure 5: Run terminal as administrator.

Task 2. Setting up environment: Node.js

Before our learning, we need to set up the relevant environments for React JS and React Native. First, we need to download [Node.JS](https://nodejs.org/en/). Actually, React is a library of JavaScript, a programming language. It is Node.JS that provides a free, cross-platform environment for running JavaScript. By clicking on the hyperlink above, we can access the website. Click on "**Download Node.js (LTS)**" to download its installation package.

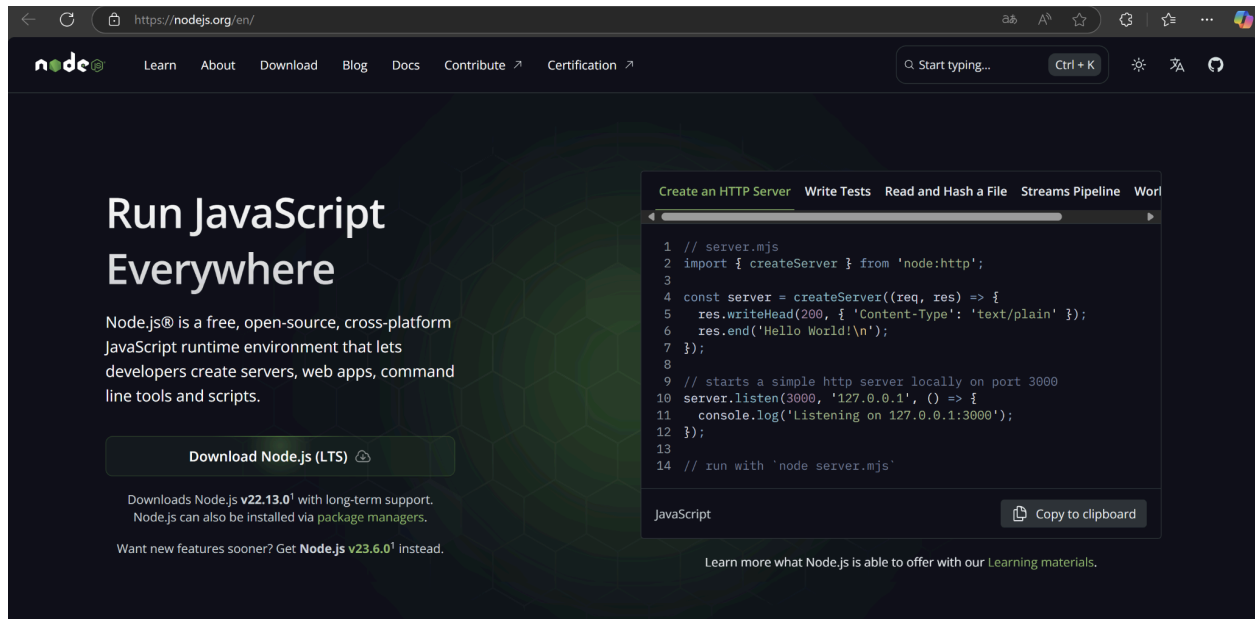


Figure 6: The official NodeJs website.

During the installation, we can simply follow the system recommended settings by clicking **Next**. After the installation is complete, open the Terminal and type:

```
node -v
```

```
npm -v
```

When you see the **version number** as shown in the Figure 7, it indicates that Node.js has been successfully installed.

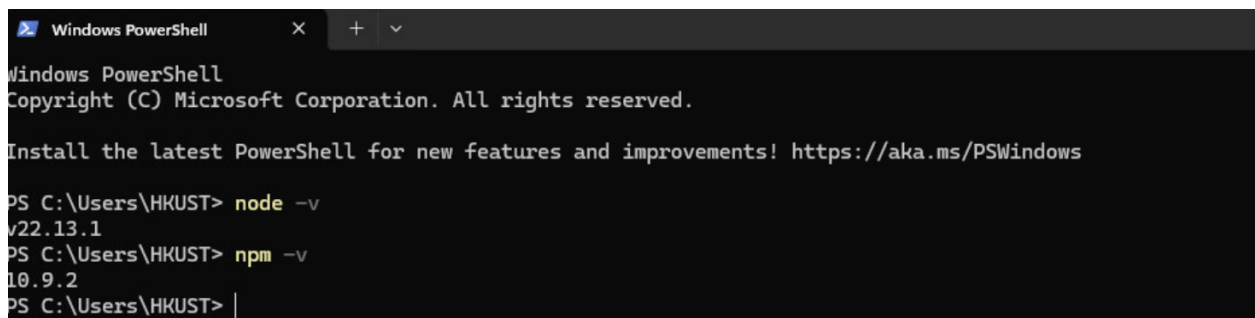


Figure 7: Verify the environment of Nodejs.

|What is npm?

| **npm** (Node Package Manager) is the package management tool for Node.js (similar to pip in Python). It is used for installing, managing, and sharing JavaScript code modules.

We can easily install existing libraries, tools, and frameworks through npm, or publish our own code for others to use.

If you encounter the following error when running `npm -v` as shown in Figure 8, you need to open the Terminal by selecting **Run as administrator**, and then enter the command:

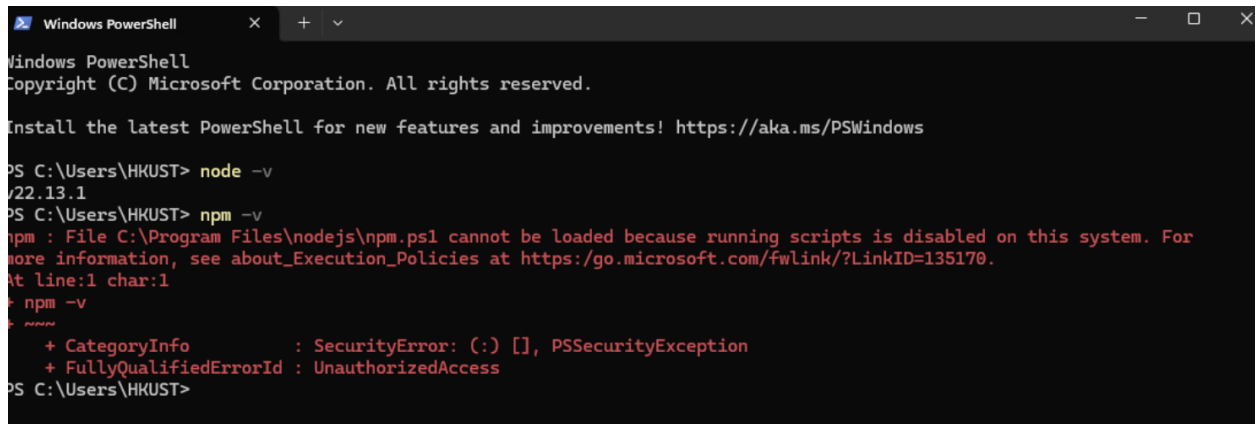
A screenshot of a Windows PowerShell terminal window. The window title is 'Windows PowerShell'. The text inside shows the user running 'node -v' which returns 'v22.13.1'. Then the user runs 'npm -v', which results in a red error message: 'npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because running scripts is disabled on this system. For more information, see about_Execution_Policies at https://go.microsoft.com/fwlink/?LinkID=135170. At line:1 char:1 * npm -v'. Below this, a red error object is displayed: '+ CategoryInfo : SecurityError: (:) [], PSSecurityException' and '+ FullyQualifiedErrorId : UnauthorizedAccess'. The prompt 'PS C:\Users\HKUST>' is visible at the bottom.

Figure 8: Potential error in Node.js setting.

`set-executionpolicy remotesigned`

This should resolve the issue.

Task 3. Setting up environment: VSCode

3.1 Install VSCode

We recommend using [VSCode \(Visual Studio Code\)](#) to edit our React code. Compared to other tools, VSCode is lightweight, supports multiple platforms, and offers extensive extensibility, making it an excellent choice for editing React code. Click the hyperlink above to visit the official website, then click **Download for Windows** to get the installation package.

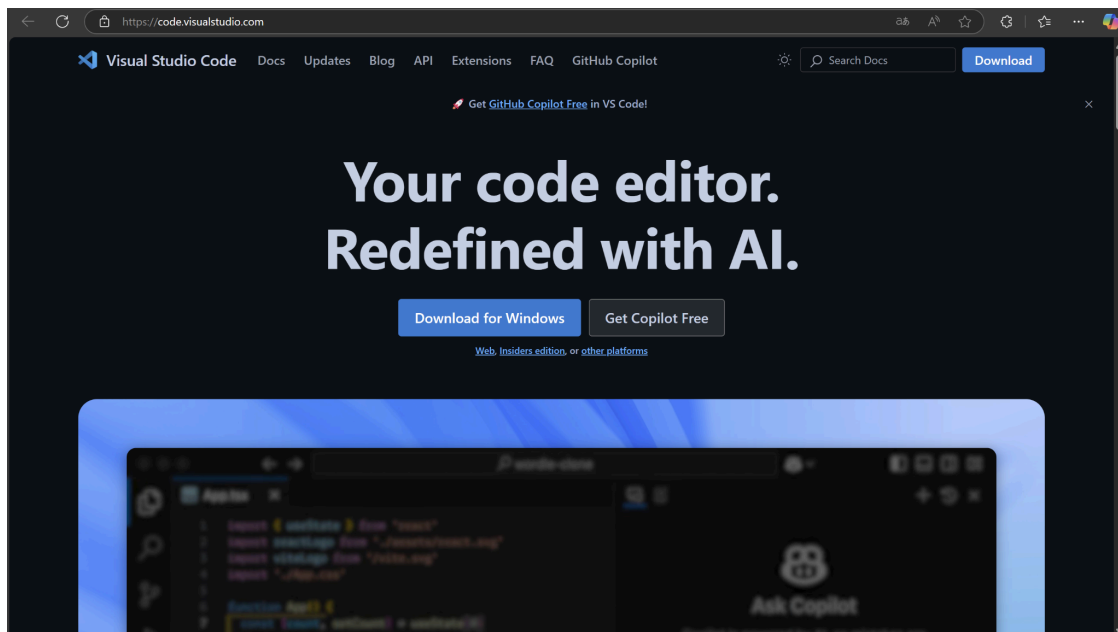


Figure 9: The official VSCode website.

3.2 VSCode Interface

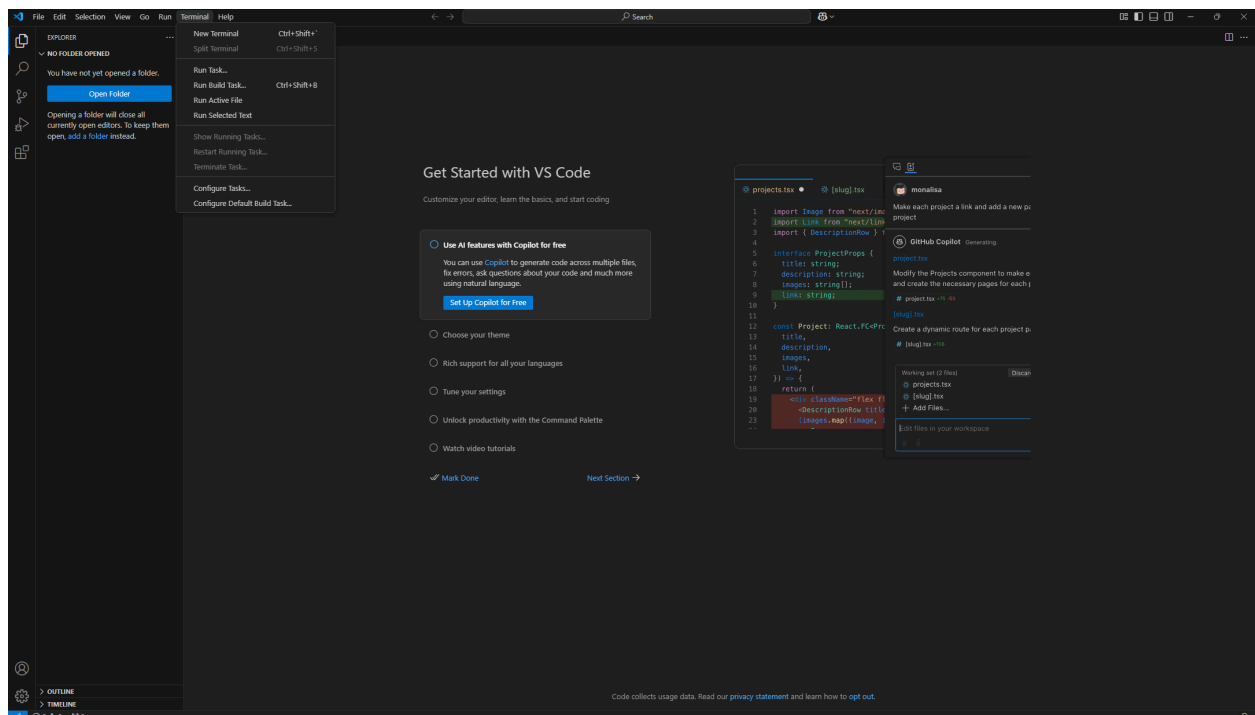


Figure 10: VSCode interface.

We open VSCode. In the interface as shown in Figure 10, the left side is the Folder view panel, where you can navigate and select files. Once a file is selected, its contents can be displayed and edited on the right side. In the top menu, click on **Terminal > New Terminal** to create a terminal that opens in the current folder. This is a feature we will frequently use in our later tasks.

VSCode is also well-known for its diverse extensions. You can click the icon in the left side (shown in Figure 11), to open the Extensions Market, where you can search for and install various extensions.



Figure 11: Extension icon in VSCode.

Task 4. Setting up environment: React JS

4.1 Create Files

Next, we can proceed to set up a React project. We start by naming our project (for example, "first_react_project"), and then we open the project via the Terminal.

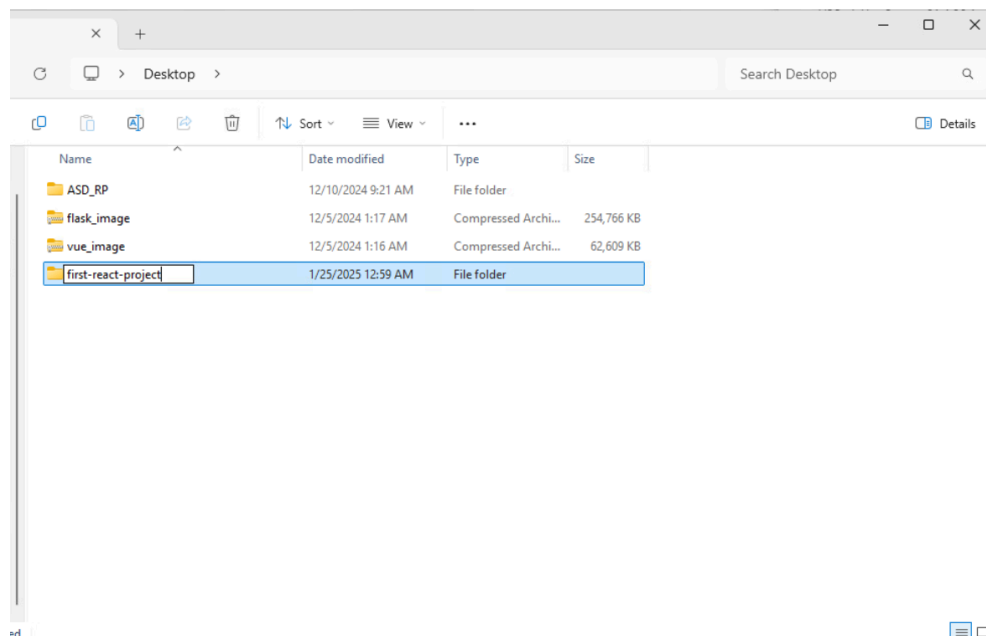


Figure 12: Create project folder in files.

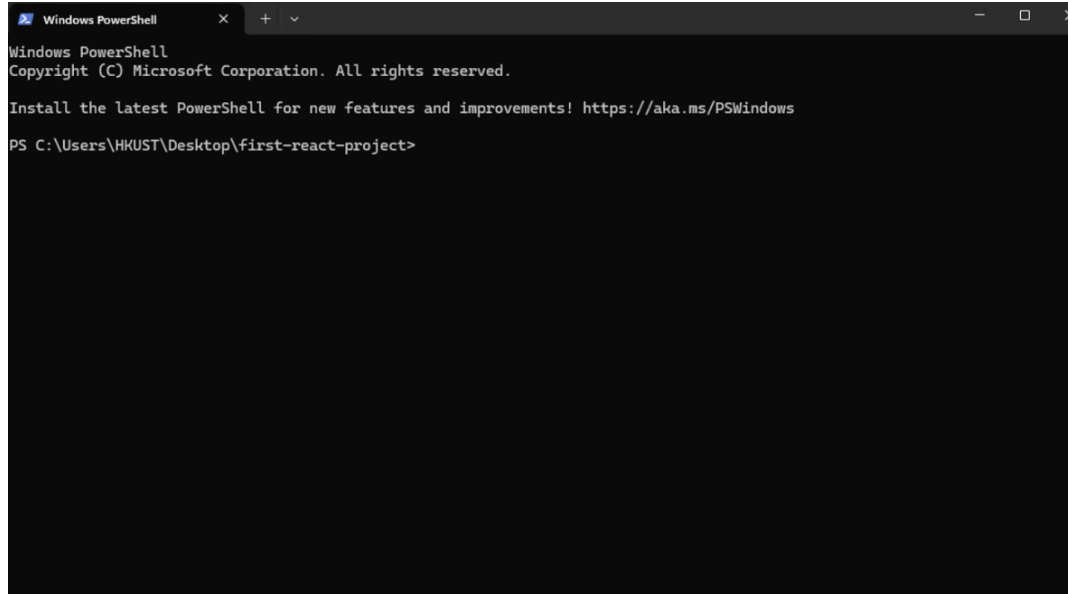


Figure 13: Open project in terminal.

4.2 Install create-react-app

React provides an official tool named `Create React App`, which is designed for swiftly setting up React projects. We can download this tool using `npm` in Terminal:

`npm install -g create-react-app`

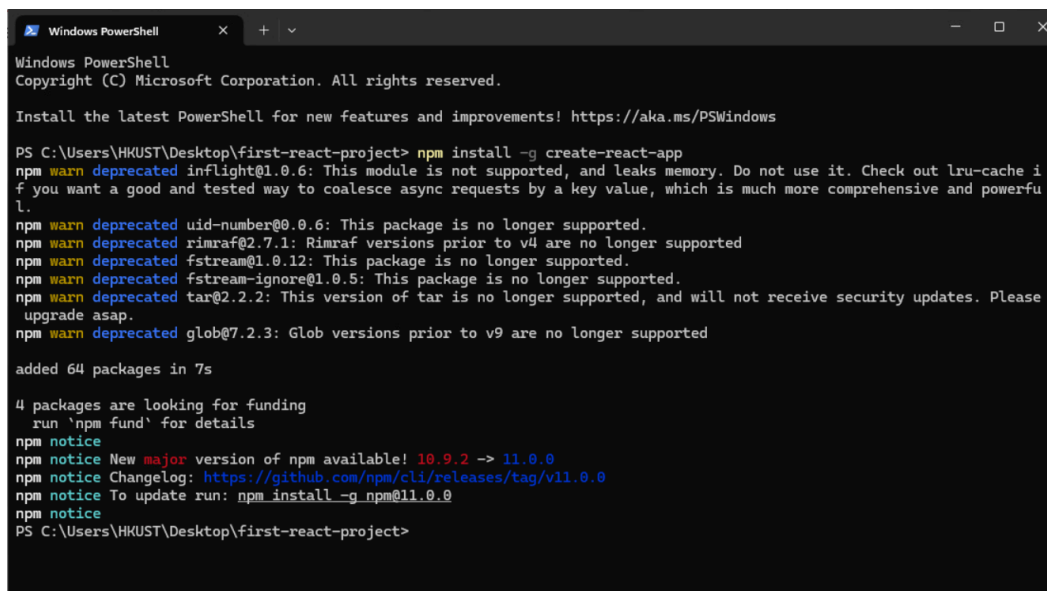


Figure 14: Install create-react-app in Terminal.

4.3 Create project

Then, execute the following command to create the project, which may take some time:

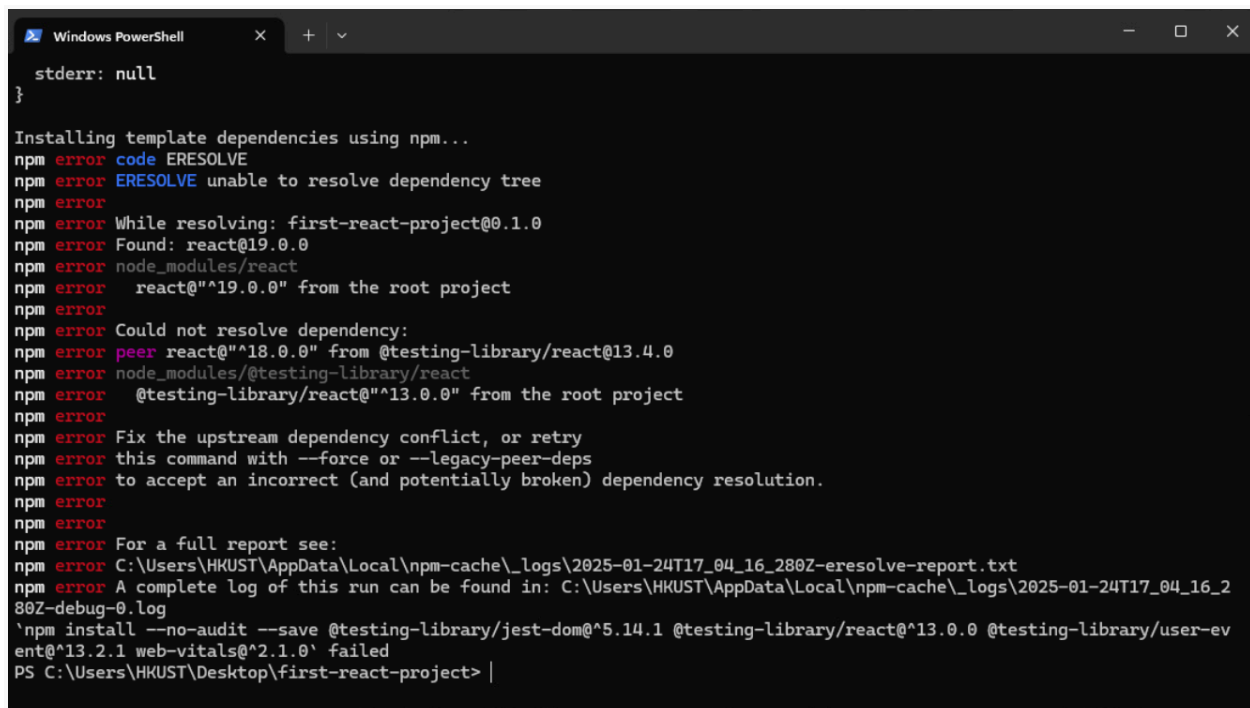
```
create-react-app first-react-project
```

```
cd first-react-project/
```

```
npm start
```

npm start is a command to run React projects. Therefore, you can apply it to run existing React projects.

In the command `create-react-app my-first-project`, if you encounter an error as shown in Figure 15, then proceed to enter the following command:



```
stderr: null
}

Installing template dependencies using npm...
npm error code ERESOLVE
npm error ERESOLVE unable to resolve dependency tree
npm error
npm error While resolving: first-react-project@0.1.0
npm error Found: react@19.0.0
npm error node_modules/react
npm error   react@"^19.0.0" from the root project
npm error
npm error Could not resolve dependency:
npm error peer react@"^18.0.0" from @testing-library/react@13.4.0
npm error   node_modules/@testing-library/react
npm error     @testing-library/react@"^13.0.0" from the root project
npm error
npm error Fix the upstream dependency conflict, or retry
npm error this command with --force or --legacy-peer-deps
npm error to accept an incorrect (and potentially broken) dependency resolution.
npm error
npm error For a full report see:
npm error C:\Users\HKUST\AppData\Local\npm-cache\_logs\2025-01-24T17_04_16_280Z-eresolve-report.txt
npm error A complete log of this run can be found in: C:\Users\HKUST\AppData\Local\npm-cache\_logs\2025-01-24T17_04_16_280Z-debug-0.log
'npm install --no-audit --save @testing-library/jest-dom@^5.14.1 @testing-library/react@^13.0.0 @testing-library/user-event@^13.2.1 web-vitals@^2.1.0' failed
PS C:\Users\HKUST\Desktop\first-react-project> |
```

Figure 15: potential error in create-react-app

```
npm install --no-audit --save @testing-library/jest-dom@^5.14.1
@testing-library/react@^13.0.0 @testing-library/user-event@^13.2.1
@types/jest@^27.0.1 @types/node@^16.7.13 @types/react@^19.0.0
@types/react-dom@^19.0.0 typescript@^4.4.2 web-vitals@^2.1.0 --legacy-peer-deps
```

Finally, the structure of the project is shown as follows:

my-first-project/

├── node_modules/

├── public/

| ├── favicon.ico

| ├── index.html

| ├── logo192.png

| ├── logo512.png

| ├── manifest.json

| └── robots.txt

├── src/

| ├── App.css

| ├── App.js

| ├── App.test.js

| ├── index.css

| ├── index.js

| ├── logo.svg

| ├── reportWebVitals.js

| └── setupTests.js

├── .gitignore

├── package.json

├── README.md

└── package-lock.json

Once the creation is complete, the system will automatically ask whether to allow networks to access this app, as shown in Figure 16. Click Allow, then it will open a web page (in my case, it's: <http://localhost:3000/>, like Figure 17). On this page, we can see the React page:

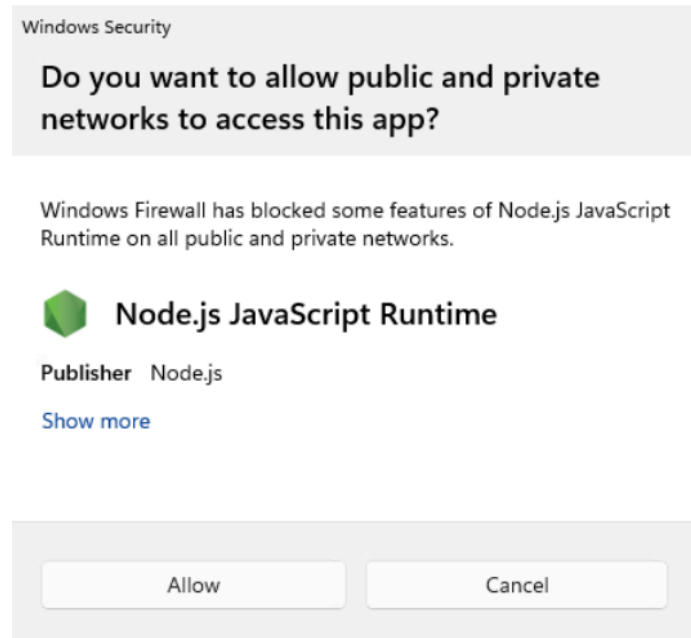


Figure 16: system request for network access (**choose Allow**).

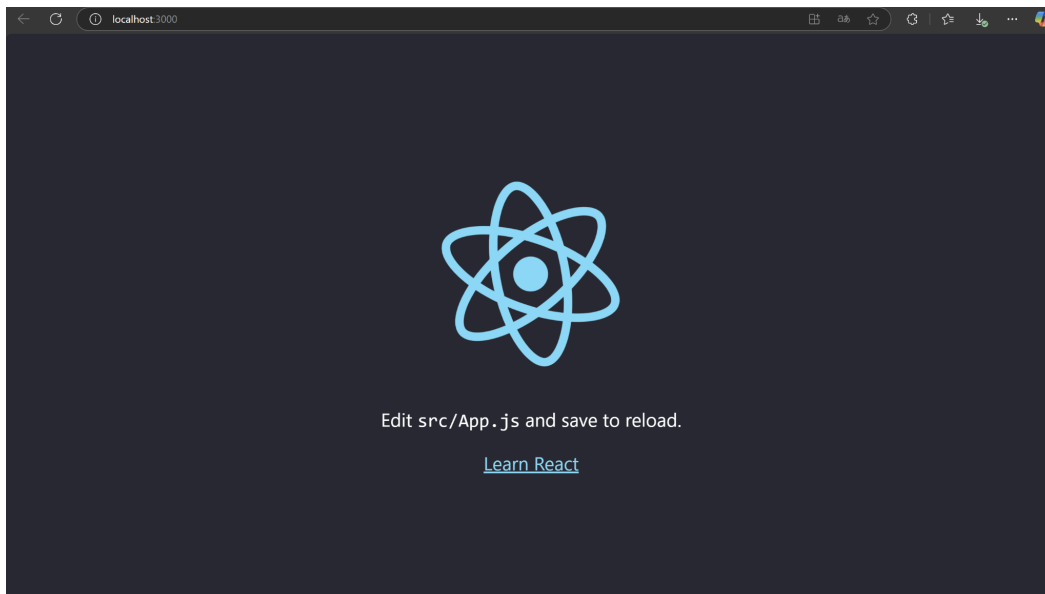


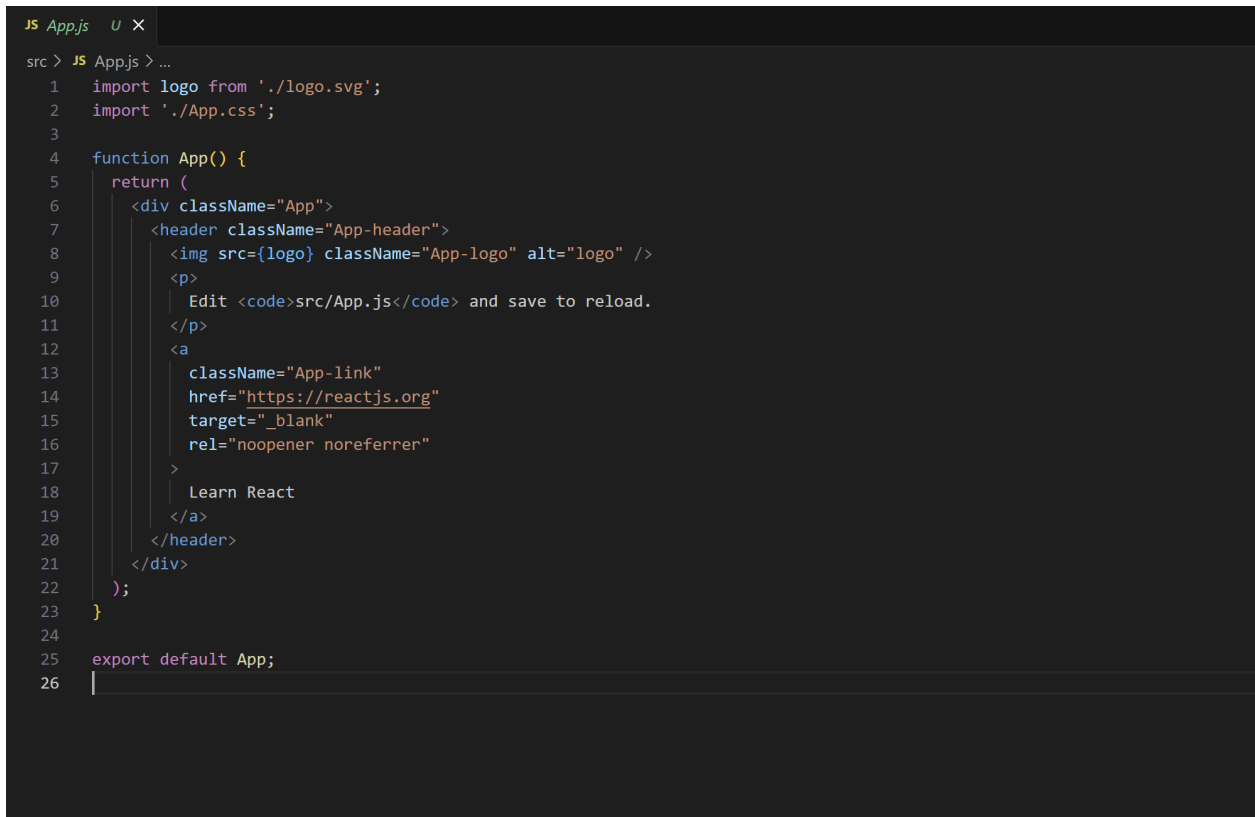
Figure 17: the page of the React.

Task 5. Edit React Code

5.1 Understand the basic structure and concepts in React.

We open it in VSCode, and within this project, there are several important files:

- `src/App.js`: The default root component where you can write your own code to build the UI.



```
JS App.js U X
src > JS App.js > ...
1  import logo from './logo.svg';
2  import './App.css';
3
4  function App() {
5    return (
6      <div className="App">
7        <header className="App-header">
8          <img src={logo} className="App-logo" alt="logo" />
9          <p>
10             Edit <code>src/App.js</code> and save to reload.
11          </p>
12          <a
13             className="App-link"
14             href="https://reactjs.org"
15             target="_blank"
16             rel="noopener noreferrer"
17           >
18             Learn React
19          </a>
20        </header>
21      </div>
22    );
23  }
24
25  export default App;
26
```

Figure 18: the code in `src/App.js`.

This code might seem a bit complex for us, so we will **remove this code and try to create our interface**. However, from this code, we can still grasp some fundamental concepts of React JS:

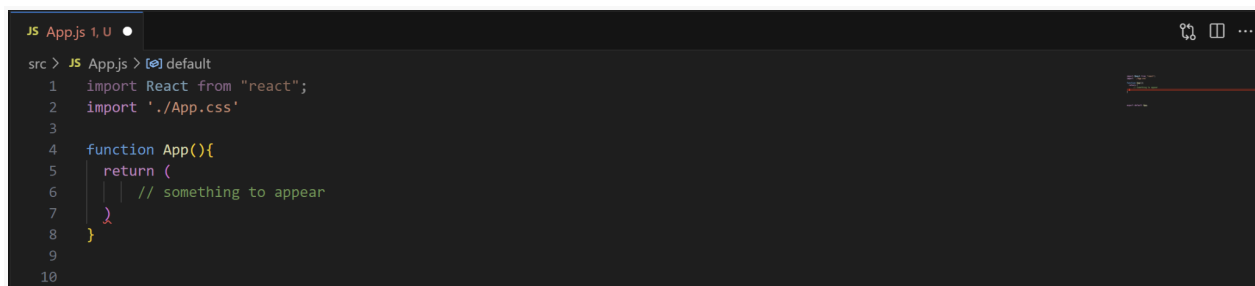
1. **Import:** At the top of the code, we can see two lines of import statements. They bring in elements from other files, allowing developers to make full use of existing code or resources.
2. **Component:** Next, we see `function App() {}`, which is a React function component named `App`. A component can be understood as an independent,

reusable UI unit. Think of it like a Lego block; by combining and arranging components, you can build a complete and varied interface.

3. **JSX Syntax:** Inside the component, we can see some commands wrapped in `<>`. They are **symmetrical**, for example, `<p>` and `</p>`, and also **hierarchical**, such as `<div>` containing `<header>`. This writing style is called JSX syntax, which can be viewed as **encapsulation of boxes**. A box (`<div>`) can contain other smaller boxes (`<p>`). Moreover, each box must have an "open" (`<div>`) and "close" (`</div>`) action, or else things will fall out.
4. **Export:** At the end of the code, there is a line `export default App;`, which means exporting this App component so that other files can import and use it. `export` is a JavaScript syntax used to expose code or components for use by other files.

5.2 Basic JSX Syntax: Edit App.js

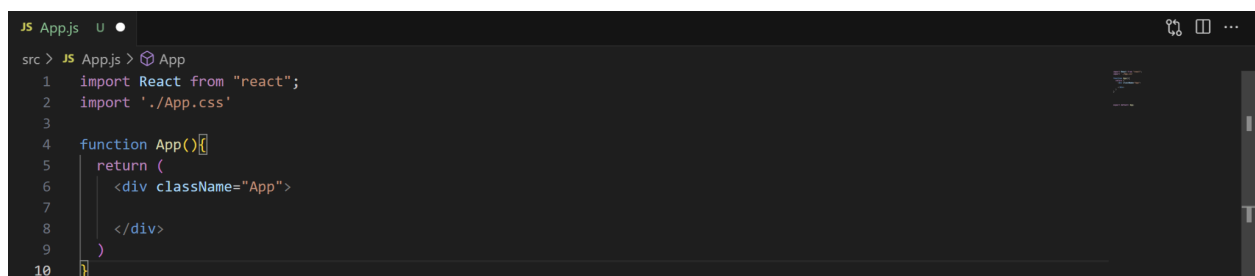
Next, we will try to delete this content and write our own function `App`. First, we import the React library to introduce React syntax and import `./App.css` to apply the website's style. Then, we construct a function `App`. **In React, all function names must start with a capital letter.** We want this `App` to return something to us when it is called:



```
JS App.js 1, U
src > JS App.js > default
1 import React from "react";
2 import './App.css'
3
4 function App(){
5   return (
6     // something to appear
7   )
8 }
9
10
```

Figure 19: Import React and construct a function `App`.

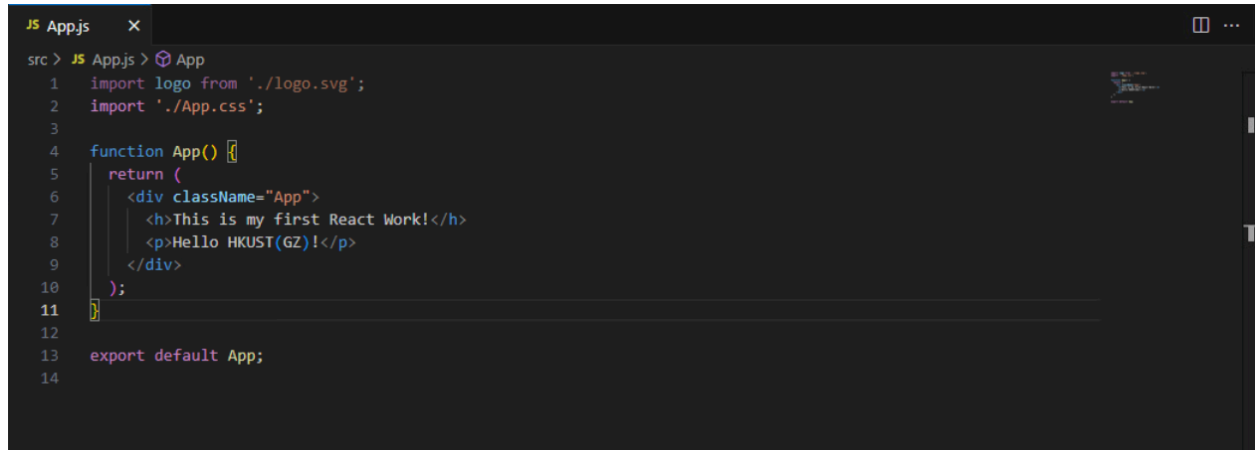
The function can only return a **single root element**. The `<div>` is one of the most common root elements, representing a division, and we name this division `App`.



```
JS App.js U
src > JS App.js > App
1 import React from "react";
2 import './App.css'
3
4 function App(){
5   return (
6     <div className="App">
7
8     </div>
9   )
10 }
```

Figure 20: Add a pair of `<div>`.

We want to write something inside the div. `<h>` and `<p>` are HTML tags used to define **text content**, representing headings and paragraphs, respectively. We will use the first line as a heading and write "This is my first React Work!", then use the second line as a paragraph and write "Hello HKUST(GZ)!".



```
src > JS App.js > App
1 import logo from './logo.svg';
2 import './App.css';
3
4 function App() {
5   return (
6     <div className="App">
7       <h>This is my first React Work!</h>
8       <p>Hello HKUST(GZ)!</p>
9     </div>
10   );
11 }
12
13 export default App;
14
```

Figure 21: Add two lines of sentences in `<div>`.

In the browser, we can now see that both lines of text we wrote are rendered:

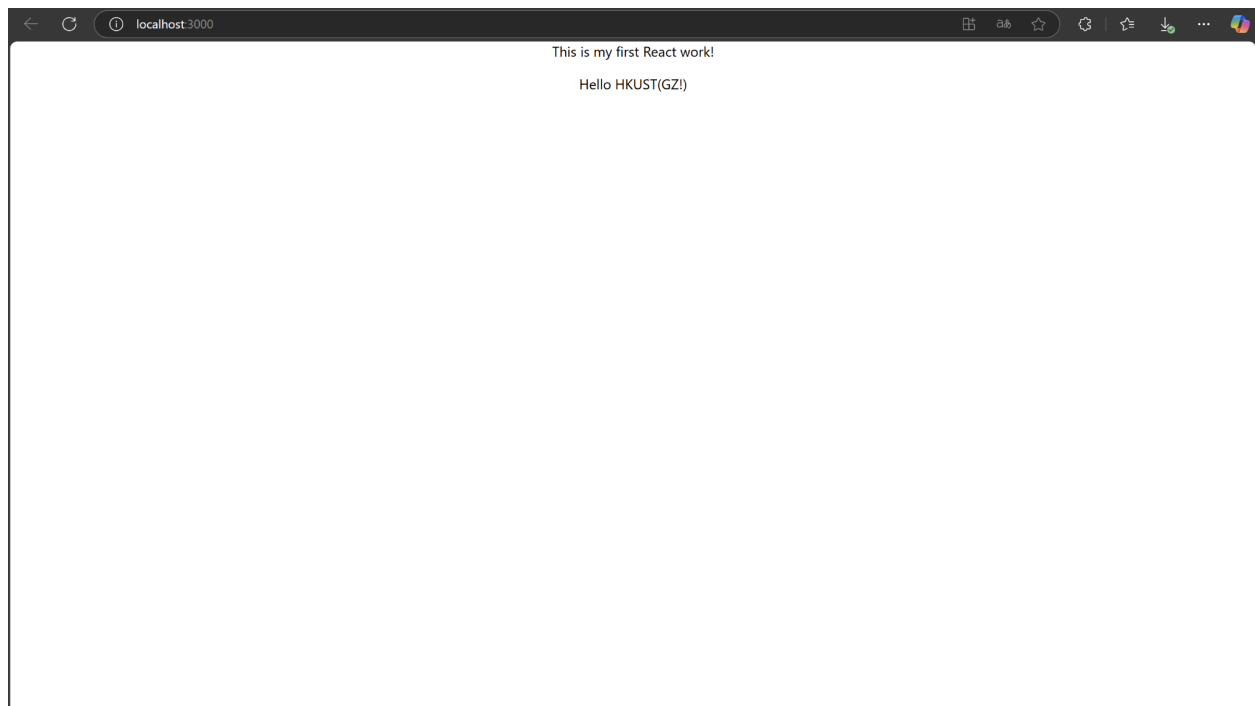
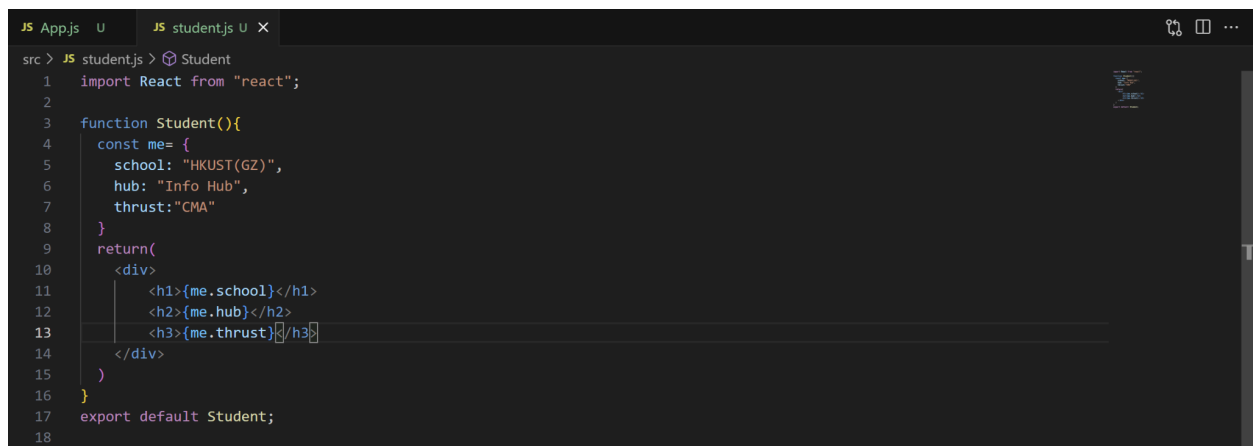


Figure 22: The page view after our editing.

5.3 Component-based Thinking

As mentioned above, components in React are like blocks that we can design and assemble to create unique pages. React's component-based approach allows developers to break down the user interface into independent, reusable components. Next, we need to display student information on the webpage, including the school, hub, and thrust. In the same folder as `App.js`, we create a `student.js` and then construct a `Student` function within it. The `Student` function contains three parts of information: `school`, `hub`, and `thrust`. When the `Student` component is called, it returns a three-level display showing the `school`, `hub`, and `thrust`. Here is how the `Student` function can be implemented. In React, `const` is commonly used to declare variables, state, functions, and other elements within components.

A screenshot of a code editor with a dark theme. The editor shows a file named 'student.js' with the following code:

```
1 import React from "react";
2
3 function Student(){
4   const me= {
5     school: "HKUST(GZ)",
6     hub: "Info Hub",
7     thrust:"CMA"
8   }
9   return(
10     <div>
11       <h1>{me.school}</h1>
12       <h2>{me.hub}</h2>
13       <h3>{me.thrust}</h3>
14     </div>
15   )
16 }
17 export default Student;
18
```

Figure 23: The `Student` function we created in `student.js`.

Afterward, we import the `Student` component in `App.js` and display it below the `Hello HKUST(GZ)!` message. We can achieve this by simply calling the `Student` component directly. Here's how it can be done:


```
JS App.js JS student.js
src > JS App.js > App
1 import React,{useState} from 'react';
2 import logo from './logo.svg';
3 import './App.css';
4 import Student from './student';
5 function App() {
6   const [count,setCount]= useState(0);
7   return (
8     <div className="App">
9       <h>This is my first React Work!</h>
10      <p>Hello HKUST(GZ)!</p>
11      <Student />
12    </div>
13  );
14 }
15
16
17 export default App;
18
```

Figure 24: Import and apply Student function in App.js.

Now, in the browser, we can see the new effect: the school, hub, and thrust are correctly displayed.

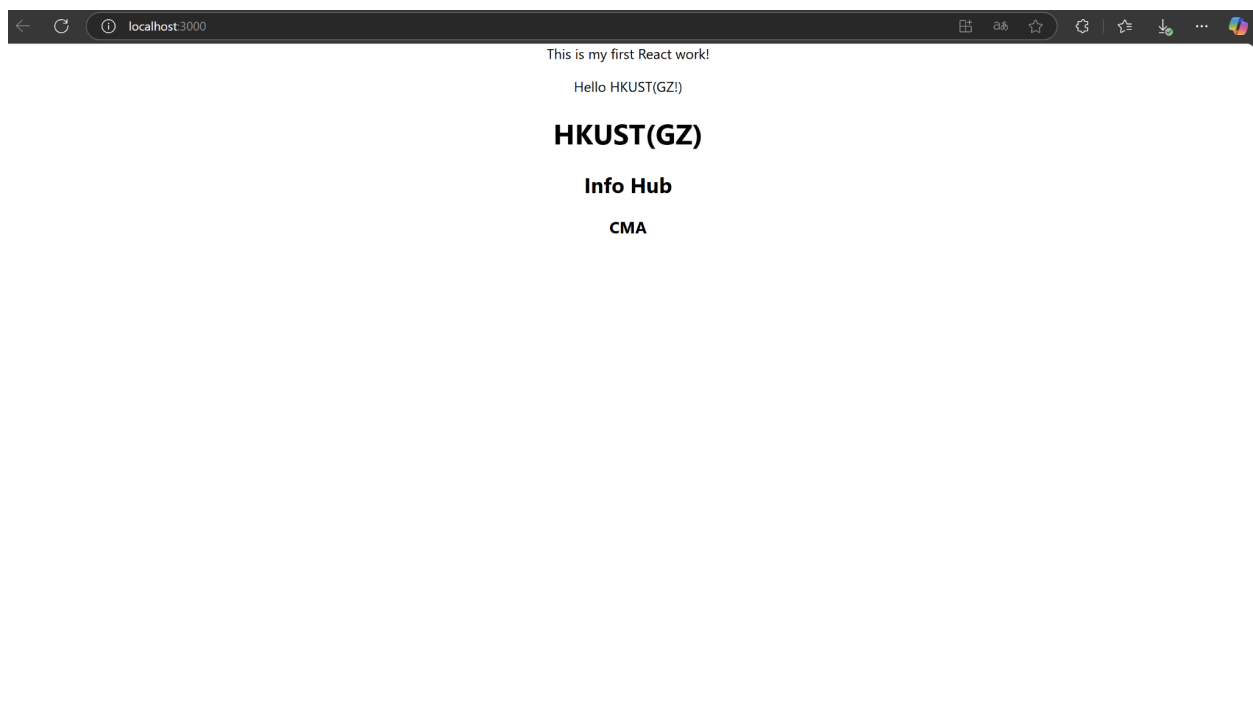


Figure 25: Student's information shown in the page.

5.4 State Change

State Change: We want this webpage to have some interactive functionality. Therefore, we introduce the `useState` Hook, which is used to add state to functional components.

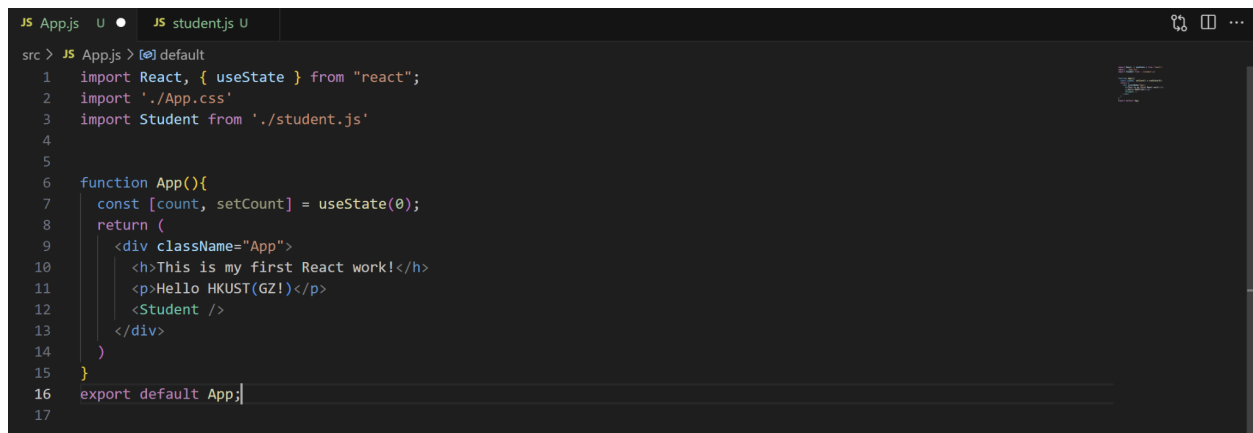
```
const [state, setState] = useState(initialValue);
```

- `state`: The current state value.
- `setState`: A function used to update the state.
- `initialValue`: The initial value of the state.

| What is Hook?

| Hook is a tool in React that allows code to remember state or perform some actions (such as logging, data fetching, reminders, etc.) at specific times. Hooks can make functions more powerful.

We try to use it in our code. First, we import `{ useState }` from React, then we set the initial value of `count` to 0 and the function to update count as `setCount`:



```
JS App.js U JS student.js U
src > JS App.js > [0] default
1  import React, { useState } from "react";
2  import './App.css'
3  import Student from './student.js'
4
5
6  function App(){
7    const [count, setCount] = useState(0);
8    return (
9      <div className="App">
10        <h>This is my first React work!</h>
11        <p>Hello HKUST(GZ!)</p>
12        <Student />
13      </div>
14    )
15  }
16  export default App;
17
```

Figure 26: Import `{useState}` and set the hook.

Next, we design a function called `handleClick`. Each time this function is triggered, it will increment the state by 1:

```
JS App.js JS student.js
src > JS App.js > App > handleClick
1 import React,{useState} from 'react';
2 import logo from './logo.svg';
3 import './App.css';
4 import Student from './student';
5 function App() {
6   const [count,setCount]= useState(0);
7   const handleClick= () => {
8     setCount(count+1)
9   }
10  return (
11    <div className="App">
12      <h>This is my first React Work!</h>
13      <p>Hello HKUST(GZ)!</p>
14      <Student />
15    </div>
16  );
17 }
18
19
20 export default App;
21
```

Figure 27: Design the function handleClick

We can see that the function here has an `=>`, which is called an **"arrow function"**. The syntax of an arrow function is:

`(parameters) => { function body }`

For example, `add = (a, b) => (a + b)` means that calling this function will return the sum of `a` and `b`. In this case, when `handleClick` is called, it will directly invoke `setCount`, ultimately increasing `count` by 1.

We also need to add a button to **trigger this function**. We use a `<button>` with the text "add" on it:

```
JS Appjs JS studentjs
src > JS Appjs > App
1 import React,{useState} from 'react';
2 import logo from './logo.svg';
3 import './App.css';
4 import Student from './student';
5 function App() {
6   const [count,setCount]= useState(0);
7   const handleClick= () => {
8     setCount(count+1)
9   };
10  return (
11    <div className="App">
12      <h>This is my first React Work!</h>
13      <p>Hello HKUST(GZ)!</p>
14      <Student />
15      <button>add</button>
16    </div>
17  );
18 }
19
20 export default App;
21
```

Figure 28: Add a trigger and a button in the interface.

To bind the function to the button, we include the `onClick` parameter in the button. `onClick` is an event-handling attribute used to handle click events. It specifies the function or action that should be executed when a user clicks on an element (such as a button, div, etc.). Here, we want to trigger `handleClick` when the button is clicked:

```
JS Appjs X JS studentjs
src > JS Appjs > App
1 import React,{useState} from 'react';
2 import logo from './logo.svg';
3 import './App.css';
4 import Student from './student';
5 function App() {
6   const [count,setCount]= useState(0);
7   const handleClick= () => {
8     setCount(count+1)
9   };
10  return (
11    <div className="App">
12      <h>This is my first React Work!</h>
13      <p>Hello HKUST(GZ)!</p>
14      <Student />
15      <button onClick={handleClick}>add</button>
16      <p>{count}</p>
17    </div>
18  );
19 }
20
21 export default App;
22
```

Figure 29: Bind trigger to the button.

Let's add `<p>{count}</p>` to display the value of `count`:

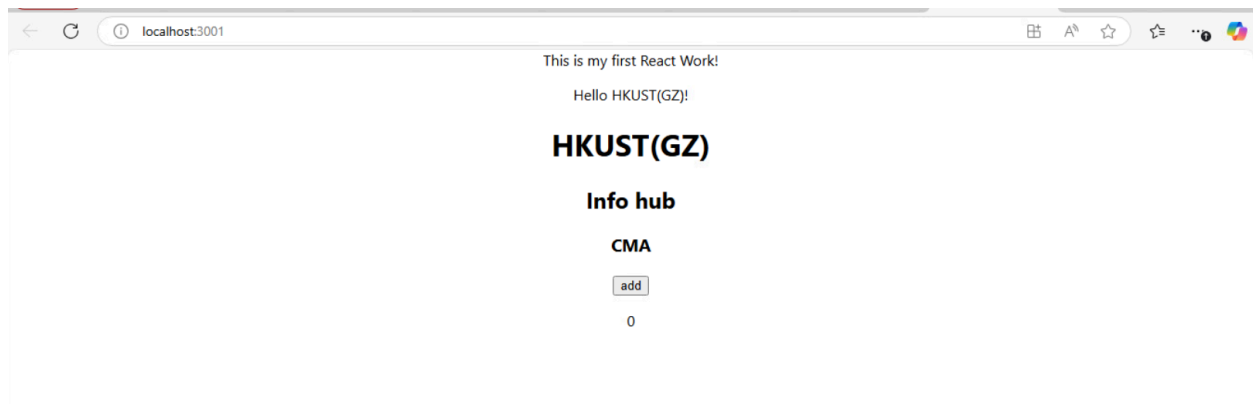


Figure 30: Final Page of React.

The final result is as shown in Figure 30. When "add" is clicked, `count` will increase by 1.

Task 6. ReactDOM

However, there might be a question here that none of us have noticed: **How does the program discover the content of function `App() {}` and then produce the corresponding changes?** Rethinking the process we have done, we exported the `Student` function in `student.js`, then imported and used this function in `app.js`. At the bottom of `app.js`, we also exported `App`, so we guess that **there is a place** where `App` is imported and used as well.

Let's open `src/index.js`. Here, we can see that the `App` is indeed imported and called! In `index.js`, there is a **ReactDOM** who renders the `App`.

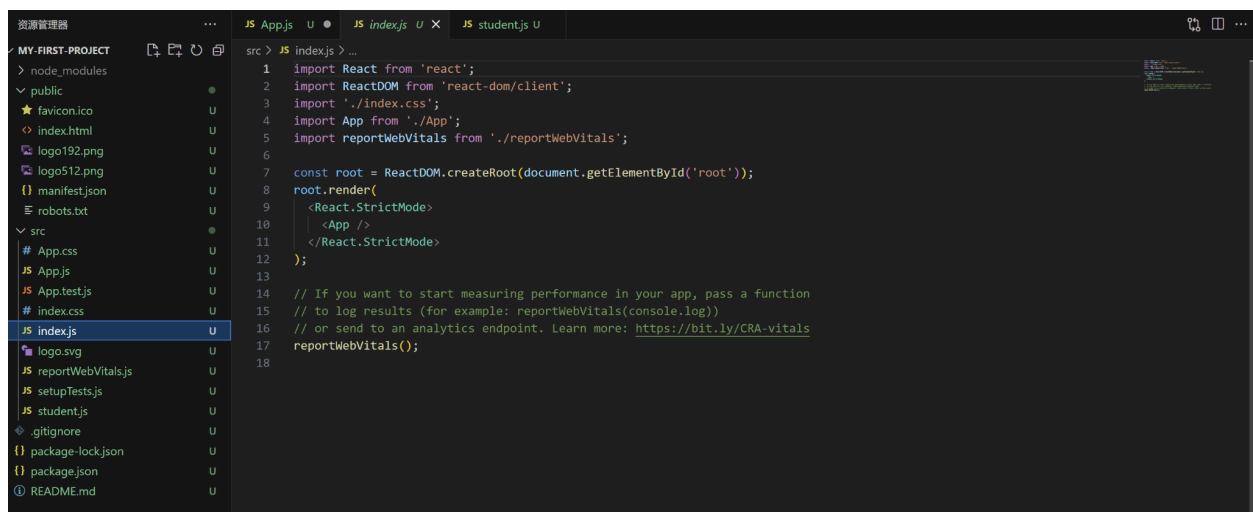


Figure 31: ReactDOM in index.js.

However, what is **ReactDOM**? Before answering this question, we need to understand **DOM** (Document Object Model). The **DOM** is a programming interface used by browsers to represent and manipulate the content of a webpage. It structures the webpage into a tree-like hierarchy, where each node represents different levels of the document (this also corresponds to the **hierarchical feature** we mentioned in JSX syntax). For example, in the code we wrote:

```
<div className="App">
  <h1>My Frist React App!</h1>
  <p>Hello HKUST(GZ)!</p>
</div>
```

Figure 32: examples of DOM

The corresponding DOM tree would look like this:

```
div (className="App")
├── h1
│   └── "My Frist React App!"
└── p
    └── "Hello HKUST(GZ)!"
```

Therefore, **ReactDOM** is responsible for rendering React components into the browser's DOM, thereby transforming changes in the React program into updates on the browser's page.

Takeaway

1. Setup Environment: Terminal
 1. Open Terminal
 2. Open File through Terminal
 3. Open Terminal as administrator
2. Setup Environment: Node.js
3. Setup Environment: VSCode
 1. Install VSCode
 2. VSCode Interface
4. Setting up environment: React JS
 1. Create Files
 2. Install create-react-app
 3. Create project
5. Edit React Code
 1. Understand the basic structure and concepts in React.
 2. Basic JSX Syntax: Edit App.js
 3. Component-based Thinking
 4. State Change
6. ReactDOM